
REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTER'ED

N/A

FUTURE WORK/APPLICATIONS

This lab introduces us to the basics of assembly for the ATXMega128A1U chip, and creates a basic decryption software through loops and jumps. This will eventually solidify our abilities to do more complex tasks, such as controlling I/O, LEDs, and more to come! Very cool indeed. 😊

PRE-LAB EXERCISES

- i. As specified in Lab 0, you should have, upon receiving your kit, verified that it contained all of the parts listed on μ PAD v2.0 Parts List (Excel or PDF). If it did not, you should have immediately notified the PI if any components were missing. For documentation purposes (and before any assembly), you should have taken pictures of all of the parts in your kit (each of the PCBs, the chips, etc.). Include these images in your Lab 1 Pre-Lab Report. Your Lab 1 report should also include images of all of your now completely constructed PCBs.

Photos attached in **APPENDIX** (Figures 1 & 2.)

- ii. Which type of memory alignment is used for program memory in the ATxmega128A1U? Byte-alignment, or word-alignment? What about for data memory?

Program memory, as stated in the accessing memory video, is ‘in the form of FLASH.’ It also uses word-alignment or two bytes/16 bits. Uses Z pointer. This lets us store ‘permanent’ data that is non-volatile. (important so we can make our program remain on a uP!) As seen in the table_load_example on the website, everything is CSEG by default, which is non-writable. (begins at 0x0000)

Data memory is in SRAM (& EEPROM for non-volatile), data memory uses byte-alignment. Uses X,Y,Z pointers. (allows read & write.) We must use .DSEG to put something in SRAM! (begins at 0x2000)

When pointing to data from program memory, we multiply our pointed to ‘word’ address by two, to find the effective byte (logical shift left).

- iii. Which assembly instructions can be used to load data indirectly from data memory within XMEGA AU microcontrollers? Which assembly instructions can be used to store data indirectly to data memory?

LD/LDD r##, X/Y/Z, loading from an address stored in a pointer a register t, with or without **post-increment/pre-decrement**, or with displacement from a register.

ST/STD X/Y/Z, r## is the storage equivalent, storing an address in a register to an address indicated by pointer. Also has **post-increment/pre-decrement**, or with displacement from a register.

- iv. Which assembly instruction can be used to load data directly from any of the general purpose I/O memory of XMEGA AU microcontrollers? Which assembly instruction can be used to store data directly to any of the I/O memory?

IN r##, 0x## allows us to read a value from address \$00 to \$FF in IO.

OUT 0x##, r## allows us to store/write a value from an address to the IO.

If we go past \$FF, we need to do extended addressing which can be done with LDS, STS, but these will take two clocks.

LDS r##, 0x##### allows us to read a value from an address in IO.

STS 0x#####, r## allows us to store/write a value from an address to the IO

- v. Which assembler directive places a byte of data in program memory? Which assembler directive allocates space within data memory? Which assembler directives allow you to provide expressions (either constant or variable) with a meaningful name?
- We would do .CSEG for program memory, and .db to allocate x bytes.
We would do .DSEG for data memory and .byte to declare a space in volatile memory as a variable.
We do .EQU for equates to make a constant variable.
We do .SET for an equate that can be changed later, as a variable.
- vi. Which assembly instructions can be used to read from (flash) program memory? For each instruction, list which registers can be used as an operand.

LPM & ELPM are the instructions we use to read from program memory. They use the Z register, and can go to any general register from r0-r31.

For ELPM we have to use RAMPZ and set it to 1, so that we can access bit 17, since with bit 16, we can only access 64/128kB of our flash. This lets us load any of the addresses available.

ELPM/LPM (assumes to load Z pointer to r0)
(E)LPM r##, Z(+)

The ending (+) allows us to post-increment Z.

- vii. Your Lab 1 report should also include images of all of your complete constructed PCBs.

Done, good sir 😊 . Check **APPENDIX** Figures 1 & 2.

- viii. In which section of program memory is address 0xFACE located?

We would be in the Application Table Section (\$F000 - \$FFFF). (Per **APPENDIX** Figure 3)

- ix. If you were to use the Memory debug window of Atmel Studio to verify that some datum was correctly stored at address 0x9D2A within program memory of the ATmega128A1U, which address would you specify within the debug window?

We want to take the 16-bit word-aligned program address and separate it into 8-bit byte-aligned program addresses, so we double 0x9D2A and get **0x13A54**.

- x. When using the internal SRAM (not EEPROM), which memory locations can be utilized for the data segment (.dseg)? Why?

We can use \$2000 - \$3FFF (Per **APPENDIX** Figure 4.)

- xi. Which is the first (i.e., lowest) program memory address (this is an address to the 16-bit wide program memory information) that would require the relevant RAMP register to be changed from its initial value of zero? Why?

In terms of 8-bit addresses, this would happen when we hit 0x10000, but since we are in words, this would be divided by two to find the effective word address, or **0x8000**. This is because once we pass 0xFFFF in data, we now need an extension to point to a higher address as four bytes will no longer be sufficient.

- xii. In the context of pointing an index to a specific program memory address within an XMEGA AU architecture, explain why and how the address value should first be altered. Similarly, in the context of pointing an index to a specific data memory address, explain why the address value should not be altered.

Because pointers are 8-bit, we want to make sure that we are showing each byte for the words in program memory, so we multiply our address by 2, for data we know that the address is already in 8-bit, so we don't change it.

PSEUDOCODE/FLOWCHARTS

Constant NULL = 0

In Program Memory:

Input_Table at 0xABBA: = [47, 0x98, 0b11110000, '#', 228, 0b10011000, 0xA2, 0b01000100, 'G', 210, '(', 0x67, 014, 0xE2, 0b11011110, 0x00]

In Data Memory:

Output_Table at 0x3744: Set Output_Table to be equal in size to Input_Table

MAIN:

Set Z to point to the start of IN_TABLE

Set X to point to the start of OUT_TABLE

LOOP:

Value = Z

Increment Z to next value in table.

If Value == Null then

 TO DONE

end

If (Bit 7 of Value is SET) then

 TO CHECK_1

ELSE

 TO FAILED_CHECK1

end

CHECK_1:

Value = Value/2

If (Value < 114) then

TO LESS_THAN_114

TO LOOP

LESS_THAN_114:

Value += 3

Value written to Pointer_X

Increment Pointer_X

TO LOOP

FAILED_CHECK1:

If (Value >= 48) then

TO GREATER_EQUAL_48

end

TO LOOP

GREATER_EQUAL_48:

Value -= 2

Value written to Pointer_X

Increment Pointer_X

TO LOOP

DONE:

TO DONE

PROGRAM CODE

```
;
; lab1.asm
;
; Created: 5/24/2026 6:45:58 PM
; Author : Evan Baessler
;
; Replace with your application code:

;*****
; File name: lab1.asm
; Author: Christopher Crary
; Last Modified By: Dr. Schwartz
; Last Modified On: 15 May 2026
; Description: To filter data stored within a predefined input table
;               based on a set of given conditions and store
;               a subset of filtered values into an output table.
;*****
;*****INCLUDES*****
.include "ATxmega128a1udef.inc"
;*****END OF INCLUDES*****
;*****EQUATES*****
; potentially useful expressions
.equ NULL = 0
.equ ThirtySeven = 3*7 + 37/3 - (3-7) ; 21 + 12 + 4 = 37, our FAVORITE NUMBER OF ALL TIME
.equ InputTableStart = 0xABBA
.equ OutputTableStart = 0x3744
;*****END OF EQUATES*****
;*****MEMORY CONFIGURATION*****
; program memory constants (if necessary)
.cseg
.org InputTableStart ; As specified, the input table starts at 0xABBA
IN_TABLE:
.db 47, 0x98, 0b11110000, '#', 228, 0b10011000, 0xA2, 0b01000100, 'G', 210
.db '(', 0x67, 0145, 0xE2
.db 0b11011110, NULL ; important lesson! must put an even amount of terms per .db or you get a padding
zero byte.
; label below is used to calculate size of input table
IN_TABLE_END:

; data memory allocation (if necessary)
.dseg
; initialize the output table starting address
.org OutputTableStart ; As specified, the output table starts at 0x3744
OUT_TABLE:
.byte (IN_TABLE_END - IN_TABLE)
;*****END OF MEMORY CONFIGURATION*****
;*****MAIN PROGRAM*****
.cseg
; configure the reset vector
; (ignore meaning of "reset vector" for now)
.org 0x0
    rjmp MAIN

; place main program after interrupt vectors
; (ignore meaning of "interrupt vectors" for now)
.org 0x100
MAIN:
; point appropriate indices to input/output tables (is RAMP needed?)
```

```
; ANSWER: yes, we need RAMP because we shift 16-bit values to the left
;           and A<<1 is greater than F, which means we will need a 16th bit
    ldi ZL, low(IN_TABLE << 1)
    ldi ZH, high(IN_TABLE << 1)
    ldi r16, byte3(IN_TABLE << 1) ; loads the overflowed value to the 3rd byte
                                   ; for RAMPZ
    out CPU_RAMPZ, r16 ; sends loaded byte3 from r16 to RAMPZ

; since our output starts at 0x3744, at it is already 8-bit, we do not shift
    ldi XL, low(OUT_TABLE)
    ldi XH, high(OUT_TABLE)

; loop through input table, performing filtering and storing conditions
LOOP:
    ; load value from input table into an appropriate register
    elpm r16, Z+ ;loads with extended addressing
    ; determine if the end of table has been reached (perform general check)
    cpi r16, NULL ; compares r16 to null, if r16 = null = 0, we branch to done
    ; if end of table (EOT) has been reached, i.e., the NULL character was
    ; encountered, the program should branch to the relevant label used to
    ; terminate the program (e.g., DONE)
    breq DONE

    ; if EOT was not encountered, perform the first specified
    ; overall conditional check on loaded value (CONDITION_1)
CHECK_1:
    ; check if the CONDITION_1 is met (bit 7 of # is set);
    ;   if not, branch to FAILED_CHECK1
    sbrc r16, 7 ; skips line below if bit 7 is SET
    rjmp FAILED_CHECK1 ; if bit 7 is not set, fail bit check and jump

    ; since the CONDITION_1 is met, perform the specified operation
    ;   (divide # by 2, unsigned)
    lsr r16
    ; check if CONDITION_1a is met (result < 114); if so, then
    ;   jump to LESS_THAN_114; else store nothing and go back to LOOP
    cpi r16, 114 ; compares r16 - 114, and sets the carry flag if r16 is less than 114
    brlo LESS_THAN_114 ; branches to LESS_THAN_114 if the compare above says so
    rjmp LOOP

LESS_THAN_114:
    ; add 3 and store the result
    subi r16, -3 ; found online that we can use the immediate subtraction with a negative
                  ; to effectively give us immediate addition, we could also load r17
                  ; with 3, and add the two using 'add', but this is much quicker!
    st X+, r16 ; stores r16 to X, post-increments X
    rjmp LOOP

FAILED_CHECK1:
    ; since the CONDITION_1 is NOT met (bit 7 of # is cleared),
    ; check if CONDITION_2b is met (result >= 48); if so, jump to
    ;   GREATER_EQUAL_48 (and do the next specified operation);
    ;   else store nothing and go back to LOOP

    cpi r16, 48
    brsh GREATER_EQUAL_48 ; branches if r16 is the same or higher than 48
    rjmp LOOP

GREATER_EQUAL_48:
    ; subtract 2 and store the result
    subi r16, 2 ; immediately subtracts 2 from r16
```

```
    st X+, r16 ; stores r16 to X, post-increments X
    ;go back to LOOP
    rjmp LOOP

; end of program (infinite loop)
DONE:
    rjmp DONE
;*****END OF MAIN PROGRAM *****
```

APPENDIX

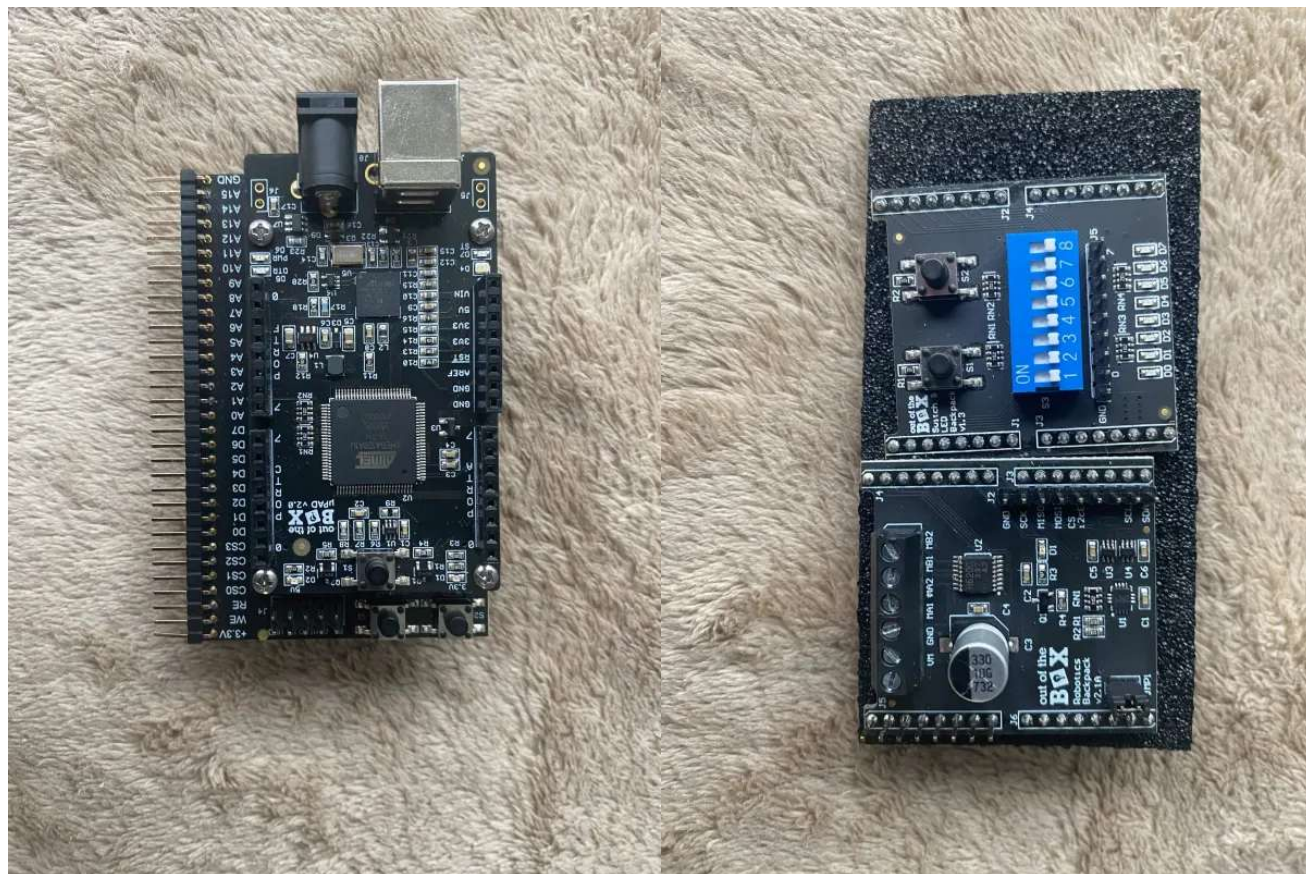


Figure 1: uPAD Development Board/Memory Base (Left), Switch & LED backpack (Top Right), Robotics Backpack (Bottom Right)

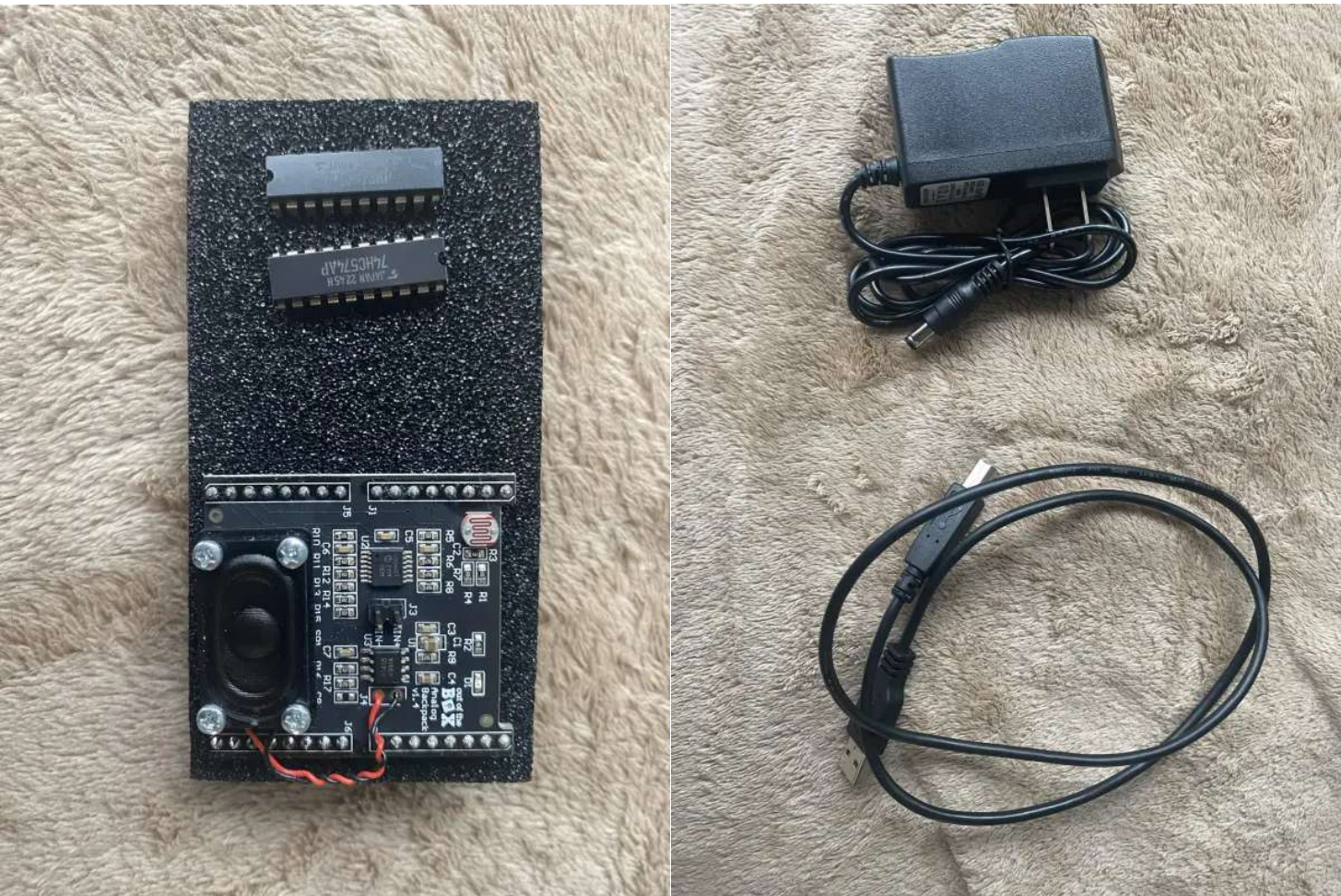


Figure 2: 74HC573 (Top Left IC), 74HC574 ('Middle' left IC), Analog Backpack (Bottom Left), 5V Power Supply (Top Right), USB Cable (Bottom Right)

Word Address		
ATxmega128A1U	ATxmega64A1U	
0	0	Application Section (bytes) (128K/64K)
		...
FFFF	/	77FF
F000	/	7800
FFFF	/	7FFF
10000	/	8000
10FFF	/	87FF
		Application Table Section (bytes) (8K/ 4K)
		Boot Section (bytes) (8K/4K)

Figure 3: Program Memory Addressing

Byte Address	ATxmega64A1U	Byte Address	ATxmega128A1U
0	I/O Registers (4K)	0	I/O Registers (4KB)
FFF		FFF	
1000		1000	
17FF	EEPROM (2K)	17FF	EEPROM (2K)
	RESERVED		RESERVED
2000	Internal SRAM (4K)	2000	Internal SRAM (8K)
2FFF		3FFF	
4000		3000	
FFFFFF	External Memory (0 to 16MB)	FFFFFF	External Memory (0 to 16MB)

Figure 4: Data Memory Addressing

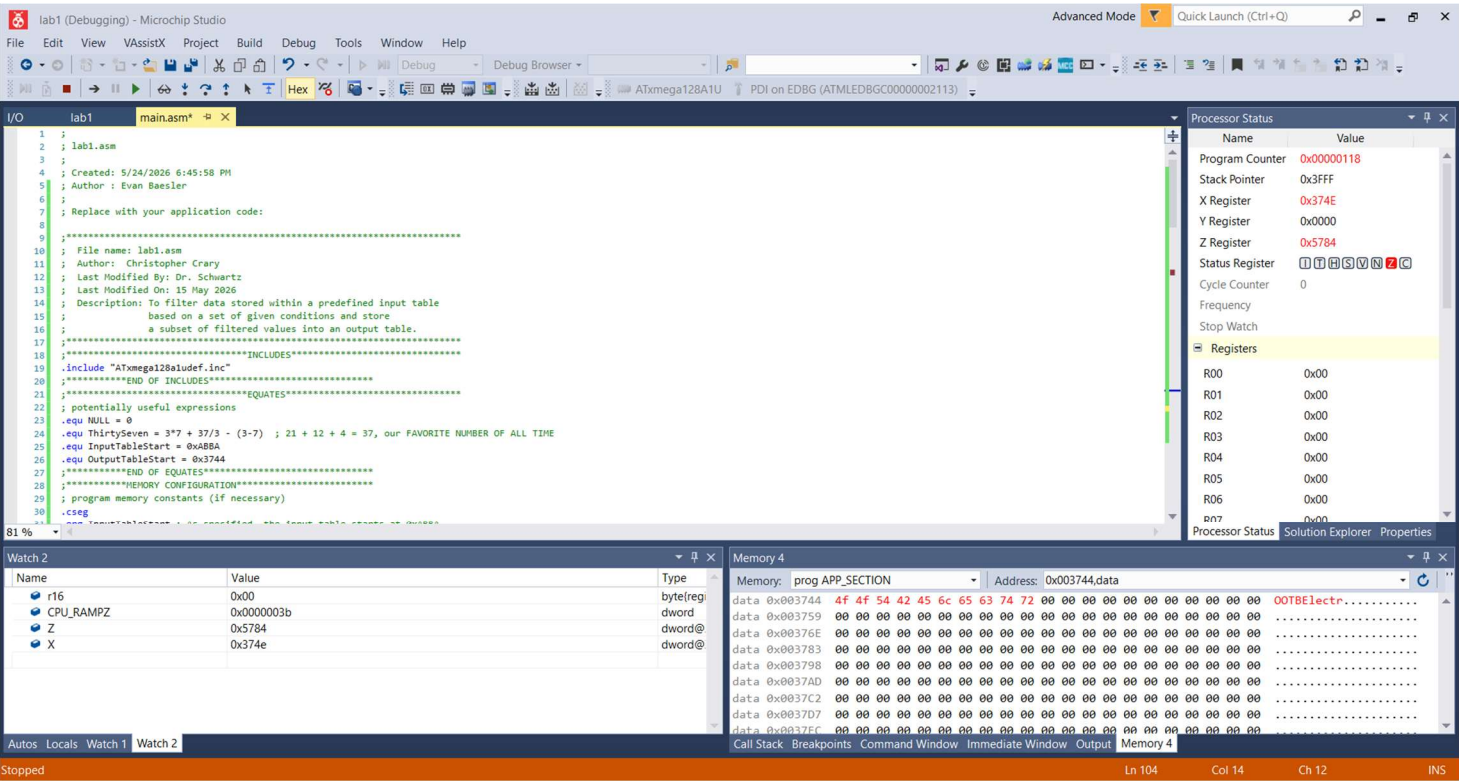


Figure 5: Hardware Simulation of our Lab 1 Script, output of OOTBElectr